

传输层、端点与 TCP

从主机交付到进程间有状态通信

408 · NET-06

1 0. 本册定位

本 Topic 回答：IP 只能把 packet 尽力送到 host，怎样进一步把数据交给正确 process，并在需要时维持一条双向、有序、流量受控的 byte-stream connection？

本册拥有 port、socket endpoint、multiplexing/demultiplexing、UDP、TCP connection identity、byte sequence state、connection lifecycle、flow control。

通用 Seq/ACK/Timer/Window 正确性由可靠传输拥有；TCP 拥塞算法由拥塞控制拥有。本册只解释它们怎样接入 TCP 状态机。

2 1. 根本问题：IP 地址只能找到机器

一台 host 同时运行浏览器、DNS resolver、邮件客户端和多个 server process。只知道 destination IP 仍无法决定把 payload 交给谁。于是传输层增加 endpoint namespace：

$$\text{Host-to-host delivery} \xrightarrow{\text{port}} \text{Process endpoint delivery}$$

发送侧 multiplexing 把多个应用的数据交给 UDP/TCP；接收侧 demultiplexing 根据 protocol 与 endpoint identifiers 找到相应 socket/connection state。

3 2. 名字与对象

- **port number**：主机内传输端点的 16-bit 标识空间；
- **socket endpoint**：至少绑定 local IP、transport protocol、local port 等本地信息的通信端点；
- **TCP connection**：由一对 endpoint 唯一确定，常表示为 (src IP, src port, dst IP, dst port)；
- **process**：使用 socket API 的执行实体，不等于 port 本身；
- **segment/datagram**：TCP/UDP 交给 IP 的传输层数据单元。

同一 server port 可以同时服务很多 TCP connections，因为 remote endpoint 不同。port 是命名维度，不是“一条连接只能占一个服务器端口”的独占资源。

4 3. UDP：保留消息边界的最小传输接口

UDP 增加 source port、destination port、length 和 checksum，然后把一个 application datagram 交给 IP。它不建立 transport connection，也不维护重传、排序、流控和拥塞窗口状态。

4.1 3.1 为什么需要 UDP

朴素方案是所有应用都使用可靠 byte stream。失败在于：

- 有些应用需要保留 message boundary;
- 有些请求很短, 不希望先建立传输连接;
- 有些应用要自己控制超时、重传、顺序或实时丢弃策略;
- DHCP 等 bootstrap 流程需要 datagram/broadcast 交互。

UDP 不是“更快的 TCP”, 而是更少策略的接口。低开销换来的代价是应用必须接受丢失、重复、乱序, 或自行构造所需语义。

4.2 3.2 UDP 首部: 四个 16-bit 字段

UDP 首部固定 8 B:

Source Port	Destination Port	Length	Checksum
-------------	------------------	--------	----------

Length 统计 UDP header 与 data 的总字节数, 故最小值为 8; 已知 IP total length 与 IP header length 时, 也可反推 UDP length。Source Port 在 IPv4 经典规范中可为 0 表示未使用, 但 Destination Port 决定接收端服务入口。

Checksum 不只覆盖 UDP header 和 data, 还概念性覆盖来自 IP 的 pseudo-header: source/destination IP、上层 protocol/next-header 与 UDP length。伪首部不实际随 UDP 重复传输, 它把“这份数据应属于哪对网络端点、哪个上层协议”纳入完整性检查, 降低误投递未被发现的风险。IPv4 UDP checksum 可选与 IPv6 中通常必需的边界必须服从题设, 不能把 IPv4 的全零语义无条件搬到 IPv6。

5 TCP 首部: 连接状态在报文中的投影

TCP 基础首部最少 20 B, 可压缩为:

Src/Dst Port	Sequence Number	Acknowledgment Number
Data Offset/Flags	Window	Checksum
Options+Padding		Data

字段应按“它暴露哪份状态”来读:

- port pair 与 IP pair 一起选择 connection;
- SEQ 标记本段首个数据字节, ACK 在 ACK flag 有效时表示下一期望字节;
- Data Offset 以 32-bit word 为单位给出 TCP header length, 最小为 5, 故最小首部 $5 \times 4 = 20$ B;
- Window 表示从 ACK number 开始、该发送者当前愿意接收的字节数, 是接收方向对反向发送者发布的流控状态;
- Checksum 覆盖 pseudo-header、TCP header 与 data, TCP 中不是可选项;
- Urgent Pointer 只有 URG 置位才有意义; Options 由 Data Offset 划定, MSS 是建连时常见选项而非固定基础字段。

408 常用 flags 可用状态事件复原: SYN 同步序号, ACK 使 acknowledgment 字段有效, FIN 表示本方向不再发送, RST 异常复位/拒绝, PSH 请求及时向应用推进, URG 使紧急指针有效。SYN 与 FIN 各占一个序号; 纯 ACK 若不带数据且无 SYN/FIN, 通常不消耗序号。字段存在不表示语义总生效, 例如 ACK number 只有 ACK flag 置位才应解释。

TCP/UDP checksum 的 pseudo-header 是跨层校验输入, 不是“传输层首部的一部分”, 也不由中

间路由器作为 TCP/UDP 字段逐跳改写。NAT 改写 IP/port 后必须相应维护校验和，正是因为这些值参与端点完整性约束。

6 5. TCP：把不可靠 packet 网络解释成可靠 byte stream

TCP 向应用提供有序 byte stream，而不是保留应用 write 的消息边界。每个传输方向都有独立序号空间和状态：

Application bytes → Byte sequence space → Segments → ACKed contiguous prefix → Receive byte stream

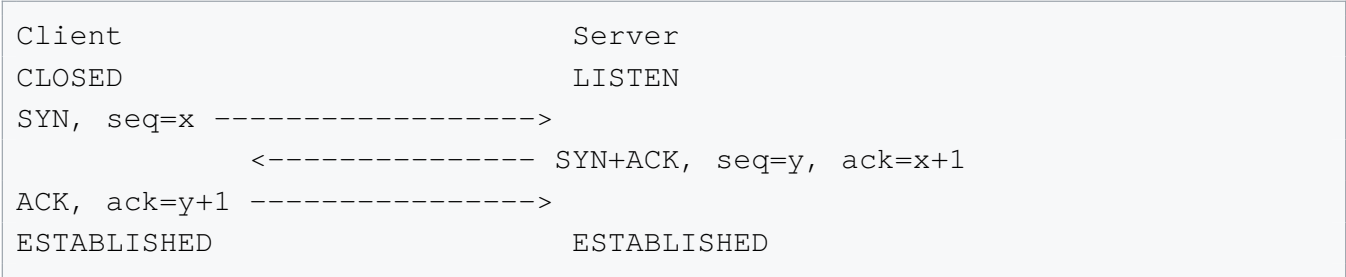
TCP sequence number 标记 segment 中第一个 data octet 的序号；ACK 通常表示“下一期望 byte”，也就是此前连续 byte 已收到。SYN 和 FIN 也占用 sequence space，这使控制事件能与数据一起被可靠排序。

7 6. 为什么连接建立需要同步双方状态

建立 TCP connection 不是“测试网络通不通”，而是让双方确认：

- 对端 endpoint 存在且愿意通信；
- 双方 initial sequence number 已知；
- 旧连接的迟到 segment 不应被误认作当前 connection 数据；
- 双向发送/接收状态可以进入 ESTABLISHED。

典型三次握手：



第三次消息的关键不是“为了凑三次”，而是让 server 知道 client 已收到 server 的 SYN/ISN。两次消息只能让 client 知道双向可达，server 仍无法确认其 SYN 已被 client 接受。

7.1 5.1 状态比报文名更重要

主动打开常见轨迹：CLOSED → SYN-SENT → ESTABLISHED。

被动打开常见轨迹：CLOSED → LISTEN → SYN-RECEIVED → ESTABLISHED。

同时打开、半开连接、RST 和重传会产生其他分支。做题必须用“收到什么 segment 前处于什么 state”推演，而不是只背三行握手图。

8 7. TCP 数据传输的三套约束

发送方在任意时刻可发送多少数据，至少同时受：

1. **可靠性状态**: 哪些 bytes 已发送、已确认、允许重传;
2. **flow control**: receiver advertised window `rwnd`;
3. **congestion control**: sender 的 `cwnd`。

教材常压缩为:

$$W_{usable} \leq \min(rwnd, cwnd) - \text{bytes in flight.}$$

`rwnd` 来自接收端 `buffer`/应用读取状态, 保护 receiver; `cwnd` 来自 sender 对 `network path` 的拥塞推断, 保护 `network`。二者数值都像“窗口”, 但状态所有者、反馈来源和失败对象不同。

9 8. Flow control: 接收端怎样表达“我还能接多少”

接收端将可用 `buffer` 空间通过 `advertised window` 反馈给发送端。应用读取数据会释放空间; 新数据到达会占用空间。基本不变量是: 发送方不能让未消费数据超过接收方声明的可接受范围。

若 receiver 通告 `zero window`, sender 暂停正常新数据发送, 但必须保留恢复探测机制, 否则 `window update` 丢失可能造成双方永久等待。

Flow control 不证明数据已可靠交付, 也不代表路径没有拥塞。它只约束 `receiver capacity`。

10 9. TCP 可靠性如何实例化通用机制

TCP 使用 `byte sequence number`、`ACK`、`retransmission timer` 和 `duplicate suppression`, 把可靠传输的机制用于 `byte stream`。

但 TCP 不能简单等同 GBN 或 SR:

- `ACK` 通常具有 `cumulative semantics`;
- receiver 可以缓存 `out-of-order bytes`;
- `retransmission` 可以由 `timeout` 或 `duplicate ACK/SACK` 等信号触发;
- 具体实现与扩展比教材中的纯 GBN/SR 状态机更复杂。

因此 GBN/SR 是理解窗口正确性和代价的模型, 不是给 TCP 贴唯一协议标签。

11 10. 连接终止: 两个方向必须分别关闭

TCP 是 `full-duplex`。一个 `endpoint` 不再发送, 并不意味着它也不能继续接收。典型主动关闭:

```
A sends FIN
-> B ACKs: A->B direction closed after prior bytes
-> B may continue sending
-> B sends FIN when its direction finishes
-> A ACKs
-> active closer waits in TIME-WAIT before state disappears
```

四次报文来自两个方向的关闭意图可能不同时发生; 若 `ACK` 与 `FIN` 可合并, 报文数可能少于四个。不能把“四次挥手”当作固定物理定律。

11.1 9.1 TIME-WAIT 维护什么

TIME-WAIT 让 active closer 有机会重发最后 ACK，并让旧 connection 的迟到 segment 在相同 end-point tuple 被复用前消失。它用暂时保留状态换取连接实例之间的不歧义。

12 11. TCP 状态的一生

每条 connection 至少维护：

- local/remote endpoint;
- send/receive initial and current sequence variables;
- send/receive window;
- retransmission and timing state;
- connection state;
- congestion-control state (由拥塞 Topic 解释)。

```
Create endpoint
-> Open / Listen
-> Synchronize sequence spaces
-> Transfer bytes in both directions
-> Retransmit / reorder / flow-control as events occur
-> Half-close or abort
-> Release state after safety interval
```

TCP 的代价不是只多 20-byte header，而是 endpoint 长期持有并更新这些 per-connection states。

13 12. 408 协议流程：UDP/TCP 逐字段推进

13.1 UDP demultiplexing 与 checksum

发送应用产生一个 datagram；UDP 加入 source port、destination port、length 和 checksum 后交给 IP。接收端先按 IP protocol 找到 UDP，再验证 length/checksum，最后以 local address/port 等绑定状态选择 endpoint；无匹配 endpoint 时不能把 payload 随意交给某个 process。UDP checksum 检查通过只保护报文完整性，不产生重传、排序或连接状态。

13.2 三次握手：同步两个序号空间

步骤	segment	Client state / knowledge	Server state / knowledge
1	SYN, seq= x	CLOSED $\rightarrow$ SYN-SENT; 占用 x	LISTEN 收到 client ISN
2	SYN+ACK, seq= y , ack= $x + 1$	知道 server 收到 x 且获知 y	SYN-RECEIVED; 等待 y 被确认
3	ACK, ack= $y + 1$	ESTABLISHED	确认 client 已收到 server ISN; ESTABLISHED

SYN 重传、同时打开和 RST 是分支；正常三行不是所有输入下的唯一轨迹。每一步都必须依据收到的 flag、SEQ、ACK 和当前 state 判断是否合法。

13.3 数据、ACK、超时与快速重传接口

TCP SEQ 指向本 segment 首个 data octet; ACK 指向下一期望 byte。收到乱序 segment 时可缓存并重复发送累计 ACK; 连续缺口被填上后 ACK 才跨过该段。RTO 到期触发可靠性重传, 并向 NET07 提供强拥塞信号; 多个 duplicate ACK 可触发 fast retransmit, 并把拥塞窗口分支交给 NET07。这里 Owns TCP 报文与连接状态, NET03 Owns 通用正确性, NET07 Owns 'cwnd' 更新。

13.4 接收窗口与 zero-window probe

接收方通告 'rwnd = receive buffer capacity - buffered unread bytes' 的题设模型。发送方正常可用量为

$$\max\{0, \min(rwnd, cwnd) - FlightSize\}.$$

若收到 zero window, 发送方暂停正常新数据, 但保留 persist/probe 分支; 探测促使接收方重报当前窗口, 避免窗口更新丢失后双方永久等待。该机制保护 receiver, 不证明 network 未拥塞。

13.5 关闭、半关闭、TIME-WAIT 与 RST

FIN 只关闭一个发送方向, 并占用一个 sequence number; 对端 ACK 后仍可在反方向发送。主动关闭方确认对端 FIN 后进入 TIME-WAIT, 以便重发最终 ACK 并隔离旧 segment。RST 表示拒绝、异常终止或不存在的连接状态, 不执行有序半关闭。题目若把 ACK 与 FIN 合并, 报文数可以少于经典四段, 但两个方向的关闭状态仍必须分别核对。

14 13. 概念边界

概念 A	≠ 概念 B	真正区别与题目信号
Port	≠ Process	port 是 transport namespace; process 是使用 endpoint
Socket endpoint	≠ TCP connection	endpoint 是一端; connection 由两端共同确定
UDP datagram	≠ TCP byte stream	UDP 保留 message boundary; TCP 只保证 byte order
Connection establishment	≠ Data reliability	握手同步 connection state; 数据可靠性需持续 ACK
ACK number	≠ Segment number	TCP ACK/SEQ 以 byte space 为核心
rwnd	≠ cwnd	receiver capacity 与 network capacity
FIN	≠ RST	FIN 有序关闭一个方向; RST 异常终止/拒绝状态
TIME-WAIT	≠ “server 固定等待”	通常由 active closer 进入, 保护最终 ACK 与旧 segment
Pseudo-header	≠ Transmitted transport header	前者只参与 checksum 计算, 字段来自 IP
UDP Length	≠ IP Total Length	前者含 UDP header+data; 后者还含 IP header

15 14. 做题调用协议

1. 写 transport protocol 和完整 endpoint tuple;
2. 明确应用需要 datagram 还是 byte stream 语义;

3. TCP 题为两个方向分别画 SEQ/ACK/state/window;
4. 先确定 SYN/FIN 是否占序号, 再计算 ACK;
5. 窗口题分开 bytes ACKed、in flight、rwnd、cwnd;
6. 建连/关闭题逐事件更新 state, 不背固定报文数;
7. 最后检查同一 server port 的不同 connections 是否被误合并。
8. 首部题先用 Data Offset/Length 划定边界, 再按 flags 判断 SEQ、ACK、Window、Urgent Pointer 是否有效;
9. checksum 题把 pseudo-header 列为计算输入, 但不得把它计入实际传输首部长度。

16 15. 贯穿母例：同一 80 端口为什么能服务多个浏览器

server 在 203.0.113.8:80 listen。两个 clients 建立：

```
(192.0.2.10:51000, 203.0.113.8:80)
(198.51.100.20:52000, 203.0.113.8:80)
```

server local port 相同, 但 remote IP/port 不同, 因此是两条独立 connection, 各自拥有 sequence、window、timer 和 state。若只看 destination port, 会把多条状态机错误地合成一条。

17 16. 高频 First Divergence

- 把 port 写成应用程序本身：没有区分 namespace 与 owner;
- TCP sequence 每 segment 加 1：忘记它以 byte 为基本坐标, SYN/FIN 另有占用;
- 三次握手解释为“第三次确认数据”：没有说明 server ISN 的确认闭环;
- 看到窗口只取 rwnd 或只取 cwnd：漏掉共同约束和 in-flight 数据;
- 把 TCP 判成 GBN：把教材可靠协议类比如成协议事实;
- 默认 client 总进入 TIME-WAIT：没有看谁主动关闭。
- 把 Data Offset 当字节数：忘记其单位是 32-bit word;
- 把 pseudo-header 画进在线 TCP/UDP 首部：混淆校验输入与传输字段;
- 看到 ACK number 就推进确认：没有先检查 ACK flag 与当前连接状态。

18 17. 一页压缩与复原问题

IP host delivery → Port demultiplexing → Endpoint pair → Sequence-space synchronization → Bidirectional byte

1. 为什么 port 不能唯一标识一条 TCP connection?
2. UDP 少掉的状态给应用带来什么自由和责任?
3. 三次握手的第三次究竟让谁知道了什么?
4. rwnd、cwnd 和 reliable sequence state 分别保护什么?
5. TIME-WAIT 怎样避免新旧 connection 混淆?
6. TCP/UDP 的 pseudo-header 为什么能发现一部分误投递, 又为什么不增加线上首部长度?

19 18. 来源与校正说明

- 对应归档中的《传输层-TCP》《传输层-传输层功能与 UDP》为空文件，因此未把空入口当作已有模型；
- TCP byte sequence、ACK、connection state 与接口边界依据 RFC 9293 校正；
- 本册保留 408 的握手、关闭、流控与端口模型，同时把现代扩展与纯 GBN/SR 类比限制在明确边界内。